```c
//Insertion and Deletion in Doubly linked list
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node* next;
    struct Node* prev;
};

struct Node* head = NULL;
void insertAtBeginning(int num)
{
    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
    new_node->data = num;
    new_node->prev = NULL;
    new_node->next = head;
    if (head != NULL)
    {
        head->prev = new_node;
    }
    head = new_node;
    printf("%d inserted at beginning of list\n", num);
}

void insertAtEnd(int num)
{
    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
    new_node->data = num;
    new_node->next = NULL;
    if (head == NULL)
    {
        new_node->prev = NULL;
        head = new_node;
        printf("%d inserted at end of list\n", num);
        return;
    }

    struct Node* current = head;
    while (current->next != NULL)
    {
        current = current->next;
    }

    current->next = new_node;
    new_node->prev = current;
    printf("%d inserted at end of list\n", num);
}
```

```c

void insertAtPosition(int num, int position)
{
    if (position < 1)
    {
        printf("Invalid position\n");
        return;
    }
    if (position == 1)
    {
        insertAtBeginning(num);
        return;
    }

    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
    new_node->data = num;

    struct Node* current = head;
    int count = 1;
    while (current != NULL && count < position)
    {
        current = current->next;
        count++;
    }

    if (count < position)
    {
        printf("Position exceeds length of list\n");
        free(new_node);
        return;
    }

    new_node->next = current;
    new_node->prev = current->prev;
    current->prev->next = new_node;
    current->prev = new_node;

    printf("%d inserted at position %d\n", num, position);
}

void deleteAtBeginning()
{
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = head;
    head = head->next;
```

```
 99:     if (head != NULL)
100:     {
101:         head->prev = NULL;
102:     }
103:     printf("%d deleted from beginning of list\n", temp->data);
104:     free(temp);
105: }
106:
107: void deleteAtEnd()
108: {
109:     if (head == NULL)
110:     {
111:         printf("List is empty\n");
112:         return;
113:     }
114:     struct Node* current = head;
115:     while (current->next != NULL)
116:     {
117:         current = current->next;
118:     }
119:     if (current == head)
120:     {
121:         head = NULL;
122:     } else {
123:         current->prev->next = NULL;
124:     }
125:
126:     printf("%d deleted from end of list\n", current->data);
127:     free(current);
128: }
129:
130: void deleteAtPosition(int position)
131: {
132:     if (head == NULL)
133:     {
134:         printf("List is empty\n");
135:         return;
136:     }
137:
138:     struct Node* current = head;
139:     int count = 1;
140:
141:     while (current != NULL && count < position)
142:     {
143:         current = current->next;
144:         count++;
145:     }
146:
147:     if (current == NULL)
```

```
148:     {
149:         printf("Position %d does not exist in list\n", position);
150:         return;
151:     }
152: 
153:     if (current == head)
154:     {
155:         head = current->next;
156:     }
157: 
158:     if (current->prev != NULL)
159:     {
160:         current->prev->next = current->next;
161:     }
162: 
163:     if (current->next != NULL)
164:     {
165:         current->next->prev = current->prev;
166:     }
167: 
168:     printf("%d deleted from position %d\n", current->data, position);
169:     free(current);
170: }
171: 
172: void display()
173: {
174:     if (head == NULL)
175:     {
176:         printf("List is empty\n");
177:         return;
178:     }
179: 
180:     struct Node* current = head;
181:     printf("List: ");
182:     while (current != NULL)
183:     {
184:         printf("%d ", current->data);
185:         current = current->next;
186:     }
187:     printf("\n");
188: }
189: 
190: void displayReverse()
191: {
192:     if (head == NULL)
193:     {
194:         printf("List is empty\n");
195:         return;
196:     }
```

```
197:     struct Node* current = head;
198:     while (current->next != NULL)
199:     {
200:         current = current->next;
201:     }
202:     printf("Reverse List: ");
203:     while (current != NULL)
204:     {
205:         printf("%d ", current->data);
206:         current = current->prev;
207:     }
208:     printf("\n");
209: }
210:
211: int main()
212: {
213:     insertAtBeginning(3);
214:     insertAtEnd(5);
215:     insertAtBeginning(7);
216:     insertAtPosition(9,2);
217:
218:     display();
219:
220:     deleteAtPosition(2);
221:
222:     display();
223:
224:     deleteAtPosition(4);
225:
226:     displayReverse();
227:
228:     return 0;
229: }
230:
```